



University College of Applied Sciences

Department of Data Science and Artificial Intelligence

Course Project: Data Mining

Project Title: Smartphone Data Scraping, Analysis, and Price Prediction

Prepared by: Shahed Salah Ferwana

Student ID: 220231087

Supervised by: Dr. Thaer Sahmoud

1. Project Overview

The objective of this project is to build an end-to-end data science pipeline that predicts smartphone prices based on their technical specifications. The project encompasses web scraping to build a custom dataset, rigorous data cleaning and imputation, feature engineering, and the training of a Machine Learning regression model.

2. Data Collection (Web Scraping)

The initial dataset was constructed by scraping e-commerce product listings (Amazon) using Python.

- **Tools Used:** `requests` for handling HTTP requests and `BeautifulSoup` for parsing HTML content.
- **Process:** The script navigated through multiple pages (pagination) to extract raw product details, including the full text descriptions and listed prices.
- **Result:** A raw dataset containing unstructured text data and raw price strings.

3. Data Cleaning and Feature Extraction

Because the scraped data was highly unstructured, Regular Expressions (Regex) were heavily utilized to extract specific features from the raw `full_details` text. To optimize the dataset for predictive modeling, only the most impactful features were retained. The final structured dataset consists of the following 6 core columns:

1. **brand:** The manufacturer of the phone (e.g., Apple, Samsung, Xiaomi).
2. **model:** The specific phone model (e.g., iPhone 17 Pro, Samsung S26).
3. **Storage:** Internal storage capacity.
4. **RAM:** Random Access Memory.
5. **Color:** The physical color of the device.
6. **Price:** The target variable (in AED).

(Note: Extraneous features such as camera megapixels, network type, and battery info were dropped during the cleaning phase to reduce dimensionality and focus on primary price drivers).

4. Advanced Missing Value Imputation

Real-world scraped data contains missing values. A strategic, multi-tiered approach was developed within the Visual Studio Code and Jupyter Notebook environments to handle these gaps without introducing bias:

- **Handling the Target Variable (Price) & Identifiers (brand):** Rows missing the `Price` or `brand` were dropped entirely. Imputing the target variable or a core identifier would introduce severe bias and invalidate model accuracy metrics.

- **Hierarchical Imputation for Technical Specs (Storage & RAM):** Missing values were filled using a three-level strategy based on the mathematical Mode:
 1. *Level 1:* Imputed using the most frequent value for that specific `model` (highest accuracy).
 2. *Level 2:* If still missing, imputed using the most frequent value for that `brand`.
 3. *Level 3:* Fallback to the global mode of the entire dataset.
- **Smart Color Imputation:** Missing colors were initially flagged as 'Unknown' to preserve rows, then later refined by imputing the most frequent color specifically associated with that particular `brand`.
- **Comprehensive Model Handling:** Missing `model` names were addressed through a smart fallback logic. If a precise model could not be extracted via Regex, the system generated a generic category combining the brand and "Other" (e.g., *Samsung Other*) to ensure complete edge-case coverage.

5. Data Transformation and Feature Engineering

To prepare the categorical and text-based data for the Machine Learning algorithm, several transformations were applied:

- **Numeric Conversion:** The `clean_to_gb` function was created to strip textual characters from the `Storage` and `RAM` columns. It standardized all capacities into pure numeric float values in Gigabytes (GB), successfully converting Terabyte (TB) values by multiplying them by 1024.
- **Target Encoding:** The `model` column contains high cardinality (many unique string values). To handle this, `TargetEncoder` was applied, replacing the categorical model names with the average target price associated with each model, providing a strong numeric signal to the algorithm.
- **One-Hot Encoding:** The `brand` and `Color` columns were converted into binary columns (0s and 1s) using `pd.get_dummies(drop_first=True)` to prevent multicollinearity.

6. Machine Learning Model Training and Evaluation

With a fully cleaned and strictly numerical dataset, a predictive model was deployed.

- **Data Splitting:** The features (X) and the target variable (y) were split into a training set (80%) and a testing set (20%) using a fixed random state to ensure reproducibility.
- **Algorithm:** A `LinearRegression` model was selected to establish the baseline relationship between a phone's core specifications and its market price.
- **Performance Metrics:** The model's predictions on the unseen test data were evaluated using standard regression metrics:
 - **Mean Absolute Error (MAE):** Measured the average absolute difference between predicted and actual prices.
 - **Root Mean Squared Error (RMSE):** Penalized larger prediction errors, providing a clear view of variance.

- **R-Squared (R^2):** Assessed the proportion of the variance in the smartphone prices that is predictable from the independent variables (Storage, RAM, Brand, Model, Color).

7. Conclusion

This project successfully demonstrates a complete data science workflow. By carefully extracting exactly six core features from unstructured web data and applying intelligent, hierarchical imputation strategies, the dataset was transformed into a robust foundation for Machine Learning. The resulting Linear Regression model effectively captures the pricing dynamics of the smartphone market based on primary technical specifications.